



UNIVERSIDAD
CATÓLICA
BOLIVIANA

UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

Carrera de Ingeniería de Sistemas
Programación II — SIS-113 | Gestión 2026

INFORME TÉCNICO

CRM de Citas para Negocios

*Sistema de gestión de citas, clientes, empleados y servicios
desarrollado íntegramente en TypeScript aplicando Programación Orientada a Objetos*

Dato	Detalle
Docente	Ing. Erwin Ramiro Flores Marca
Integrantes	Luis Fabricio Rivero Aban Santiago Antonio Encinas Jadue Santiago Zamora Barrios
Materia	Programación II (SIS-113) — Paralelo 1
Repositorio	github.com/FabricioRivero/crm-final-sis113
Fecha de entrega	1 de julio de 2026

Índice

1. Resumen Ejecutivo
2. Descripción del Sistema
3. Arquitectura y Diseño POO
4. Enums e Interfaces
5. Genéricos y Repositorio
6. Manejo de Excepciones
7. Pruebas Unitarias
8. Estructura del Proyecto
9. Instrucciones de Ejecución
10. Capturas de Pantalla (Interfaz)
11. Desafíos y Soluciones
12. Conclusiones
13. Referencias Bibliográficas

1. Resumen Ejecutivo

El presente informe documenta el desarrollo del proyecto “CRM de Citas para Negocios”, un sistema de gestión de relaciones con clientes orientado específicamente a la administración de citas, reservas y agendas. El proyecto fue elaborado como trabajo final de la materia Programación II (SIS-113) de la Universidad Católica Boliviana “San Pablo”, y responde a la necesidad práctica de pequeños y medianos negocios de servicios —barberías, salones de belleza, consultorios médicos y talleres técnicos— de contar con una herramienta digital que centralice el registro de clientes, empleados, servicios y citas, sustituyendo las agendas manuales y las hojas de cálculo dispersas.

El sistema fue construido íntegramente en TypeScript, lo que permitió aplicar de forma rigurosa los principios de la Programación Orientada a Objetos (POO) exigidos en la materia: abstracción, herencia, polimorfismo, encapsulamiento, interfaces y genéricos, además de un manejo estructurado de excepciones. El diseño se organiza en torno a una jerarquía de clases encabezada por la clase abstracta *Persona*, de la cual heredan *Cliente* y *Empleado*, junto con las clases *Servicio* y *Cita*, que modelan respectivamente el catálogo de servicios ofrecidos y el ciclo de vida de una cita (pendiente, confirmada, completada o cancelada).

Como elemento central de la arquitectura de persistencia, el proyecto implementa una clase genérica *Repositorio<T>*, capaz de almacenar, recuperar, actualizar y eliminar cualquier entidad del sistema que cumpla con un contrato mínimo (interfaz *IEntidad*), reutilizando una única implementación para gestionar clientes, empleados y servicios sobre *localStorage* del navegador, sin necesidad de un servidor backend. La calidad del código se respalda con una suite de pruebas unitarias propia, organizada en cinco bloques temáticos que cubren herencia y polimorfismo, enums, transiciones de estado, métodos estáticos/validaciones y el repositorio genérico.

En conjunto, el proyecto demuestra la aplicación integral de los conceptos de POO revisados durante el curso, materializados en un producto funcional, con interfaz web propia (HTML5, CSS3 y TypeScript compilado a JavaScript ES2020), disponible públicamente en el repositorio de GitHub del equipo y ejecutable sin dependencias externas más allá de Node.js para el proceso de compilación y pruebas.

2. Descripción del Sistema

2.1. ¿Qué es un CRM de citas?

Un CRM (Customer Relationship Management) de citas es un sistema de software cuyo propósito es administrar el ciclo completo de la relación entre un negocio y sus clientes en torno a la programación de servicios: desde el registro del cliente y la definición del catálogo de servicios, hasta la asignación de un empleado, la programación de la cita en una fecha y hora determinadas, y el seguimiento de su estado hasta su finalización. A diferencia de un CRM genérico de ventas, un CRM de citas está centrado en la variable tiempo (disponibilidad de agenda) y en la relación tripartita cliente-empleado-servicio.

2.2. Negocios a los que aplica

El modelo de datos del sistema fue diseñado de forma suficientemente genérica para adaptarse a distintos rubros de servicios con agenda, entre ellos:

- Barberías y peluquerías (cortes, coloración, tratamientos capilares).
- Salones de belleza y centros de estética.
- Consultorios médicos y clínicas (consultas y citas de especialidad).
- Talleres técnicos y de mantenimiento (servicios programados).

2.3. Funcionalidades por módulo

La siguiente tabla resume los módulos funcionales implementados en la interfaz del sistema:

Módulo	Descripción funcional
Gestión de Clientes	Registro, búsqueda, historial de citas asociadas y datos de contacto del cliente.
Gestión de Citas	Creación, confirmación, cancelación y finalización ("completado") de citas.
Gestión de Servicios	Catálogo de servicios con precio, duración en minutos y categoría.
Gestión de Empleados	Registro de especialidades del personal y control de disponibilidad.
Dashboard	Estadísticas generales del negocio y listado de próximas citas.
Calendario	Vista mensual de citas con navegación entre meses.

3. Arquitectura y Diseño POO

La arquitectura del sistema se organiza en dos capas principales dentro de la carpeta `src/`: `models/`, que contiene las clases de dominio y sus reglas de negocio, y `services/`, que contiene la clase genérica encargada de la persistencia. Esta separación aísla las reglas de negocio (validaciones, transiciones de estado, cálculo de tipo de cliente, etc.) de la infraestructura de almacenamiento, de modo que el mismo `Repositorio<T>` pueda reutilizarse para cualquier entidad sin modificar su código.

3.1. Diagrama de clases (UML)

El diagrama siguiente resume la estructura de clases del sistema y sus relaciones. `Persona` es la superclase abstracta de la que heredan `Cliente` y `Empleado` (herencia y polimorfismo); `Servicio` implementa el contrato definido por la interfaz `IServicio`; `Cita` relaciona por identificador a un `Cliente`, un `Empleado` y un `Servicio`, y gestiona su propio ciclo de estados; y `Repositorio<T>` es una clase genérica, independiente del dominio, que puede administrar instancias de `Cliente`, `Empleado` o `Servicio` en tanto estas implementen la interfaz mínima `IEntidad`.

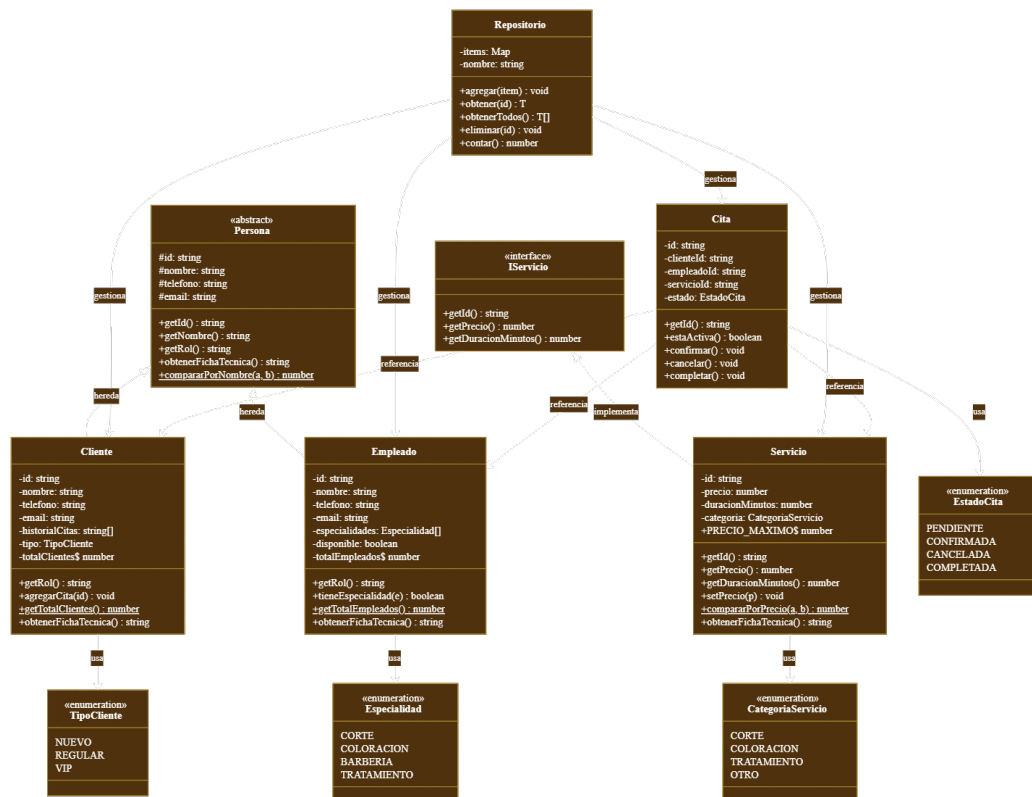


Figura 1. Diagrama de clases del sistema CRM de Citas (`docs/uml_crm_citass.png`).

3.2. Tabla de clases y conceptos POO aplicados

Clase	Tipo	Concepto POO principal	Descripción
Persona	Abstracta	Abstracción / Herencia	Clase base con id, nombre, teléfono y email;

Clase	Tipo	Concepto POO principal	Descripción
			declara getRol() y obtenerFichaTecnica() como métodos abstractos.
Cliente	Concreta	Herencia / Polimorfismo	Extiende Persona; agrega historial de citas, notas y un tipo (Nuevo, Regular, VIP) que evoluciona automáticamente.
Empleado	Concreta	Herencia / Polimorfismo	Extiende Persona; agrega especialidades (enum) y estado de disponibilidad.
Servicio	Concreta	Interfaces / Encapsulamiento	Implementa IServicio; expone precio, duración y categoría con validaciones en el constructor.
Cita	Concreta	Encapsulamiento / Excepciones	Gestiona el ciclo de vida de una cita mediante un enum de estados y métodos que validan transiciones válidas.
Repositorio<T>	Genérica	Genéricos	Repositorio<T> extends IEntidad> reutilizable para cualquier entidad, con persistencia en localStorage.

3.3. Abstracción y herencia — clase Persona

La clase Persona no puede instanciarse directamente: su constructor verifica new.target y lanza una excepción si se intenta crear una Persona “pura”, forzando a que solo sus subclases concretas (Cliente, Empleado) puedan construirse.

```
export abstract class Persona {
  protected readonly id: string;
  protected nombre: string;
  protected telefono: string;
  protected email: string;

  constructor(id: string, nombre: string, telefono: string, email: string) {
    if (new.target === Persona) throw new Error('Persona es una clase abstracta');
    if (!id.trim()) throw new Error('El ID no puede estar vacío');
    this.id = id;
    this.nombre = nombre;
    this.telefono = telefono;
    this.email = email;
  }

  // Metodo abstracto - polimorfismo
  abstract getRol(): string;
  abstract obtenerFichaTecnica(): string;

  // Metodo estatico - miembro de clase
  static compararPorNombre(a: Persona, b: Persona): number {
    return a.getNombre().localeCompare(b.getNombre());
  }
}
```

```

    }
  }

```

Figura 2. `src/models/Persona.ts` — clase abstracta base (fragmento).

3.4. Herencia y polimorfismo — Cliente y Empleado

Cliente y Empleado extienden `Persona` con `super(...)` y sobrescriben `getRol()` de forma distinta, de modo que un mismo arreglo `Persona[]` puede contener ambos tipos y responder de forma polimórfica al invocar `getRol()` u `obtenerFichaTecnica()`.

```

export class Cliente extends Persona {
  private historialCitas: string[];
  private tipo: TipoCliente;
  private static totalClientes: number = 0;

  constructor(id: string, nombre: string, telefono: string, email: string,
    notas: string = '', tipo: TipoCliente = TipoCliente.NUEVO) {
    if (!nombre.trim()) throw new Error('El nombre no puede estar vacío');
    if (!telefono.trim()) throw new Error('El teléfono no puede estar vacío');
    super(id, nombre, telefono, email);
    this.historialCitas = [];
    this.tipo = tipo;
    Cliente.totalClientes++;
  }

  getRol(): string { return 'Cliente'; } // Polimorfismo

  agregarCita(citaId: string): void {
    if (!citaId) throw new Error('El ID de cita no puede estar vacío');
    this.historialCitas.push(citaId);
    if (this.historialCitas.length >= 10) this.tipo = TipoCliente.VIP;
    else if (this.historialCitas.length >= 1) this.tipo = TipoCliente.REGULAR;
  }
}

```

Figura 3. `src/models/Cliente.ts` — herencia de `Persona` y actualización automática de tipo (fragmento).

De forma análoga, `Empleado` hereda de `Persona` e implementa `getRol()` devolviendo 'Empleado', gestionando además un arreglo de especialidades (enum `Especialidad`) y un indicador de disponibilidad:

```

export class Empleado extends Persona {
  private especialidades: Especialidad[];
  private disponible: boolean;

  constructor(id: string, nombre: string, telefono: string, email: string,
    especialidades: Especialidad[]) {
    if (especialidades.length === 0)
      throw new Error('El empleado debe tener al menos una especialidad');
    super(id, nombre, telefono, email);
  }
}

```

```
    this.especialidades = especialidades;
    this.disponible = true;
  }

  getRol(): string { return 'Empleado'; } // Polimorfismo
  tieneEspecialidad(esp: Especialidad): boolean {
    return this.especialidades.includes(esp);
  }
}
```

Figura 4. `src/models/Empleado.ts` — segunda subclase concreta de *Persona* (fragmento).

3.5. Encapsulamiento

Todas las clases del dominio declaran sus atributos como `private` o `protected`, exponiendo el estado únicamente a través de métodos `getX()/setX()` que, además, validan la información antes de modificarla (por ejemplo, `setNombre()` y `setTelefono()` en *Persona* rechazan cadenas vacías), evitando que el estado interno del objeto quede en un estado inconsistente.

4. Enums e Interfaces

El sistema define cuatro enumeraciones que restringen a un conjunto cerrado y auto-documentado los valores válidos para conceptos clave del dominio, evitando el uso de cadenas de texto “mágicas” dispersas por el código.

Enum	Archivo	Valores	Propósito
EstadoCita	Cita.ts	PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA	Controla el ciclo de vida de una cita y las transiciones permitidas.
TipoCliente	Cliente.ts	NUEVO, REGULAR, VIP	Clasifica automáticamente al cliente según su historial de citas.
Especialidad	Empleado.ts	CORTE, COLORACION, BARBERIA, TRATAMIENTO, ESTETICA, MEDICO, TECNICO	Define las especialidades que puede tener un empleado.
CategoriaServicio	Servicio.ts	CORTE, COLORACION, TRATAMIENTO, ESTETICA, MEDICO, TECNICO, OTRO	Clasifica los servicios del catálogo del negocio.

```
export enum EstadoCita {
  PENDIENTE = 'pendiente',
  CONFIRMADA = 'confirmada',
  CANCELADA = 'cancelada',
  COMPLETADA = 'completada'
}
```

Figura 5. src/models/Cita.ts — enum EstadoCita, base de la máquina de estados de la cita.

4.1. Interfaz IServicio

La interfaz IServicio define el contrato mínimo que debe cumplir cualquier implementación de un servicio del catálogo, desacoplando el resto del sistema de la implementación concreta de la clase Servicio. Esto permite, por ejemplo, que el método estático Servicio.compararPorPrecio() opere sobre el tipo IServicio en lugar de la clase concreta.

```
// Interface - elemento de competencia 3
export interface IServicio {
  getId(): string;
  getNombre(): string;
  getPrecio(): number;
  getDuracionMinutos(): number;
  getCategoria(): CategoriaServicio;
}

export class Servicio implements IServicio {
```

```
/* ... implementación concreta ... */  
}
```

Figura 6. *src/models/Servicio.ts* — interfaz *IServicio* implementada por *Servicio*.

5. Genéricos y Repositorio

El requisito de genéricos se resuelve mediante la clase `Repositorio<T>` extends `IEntidad`, definida en `src/services/Repositorio.ts`. El parámetro de tipo `T` queda restringido (bounded generic) a cualquier entidad que implemente la interfaz `IEntidad` (`getId()` y `toJSON()`), lo que permite que el repositorio ofrezca operaciones CRUD completamente tipadas sin necesidad de duplicar código para `Cliente`, `Empleado` o `Servicio`.

```
export interface IEntidad {
  getId(): string;
  toJSON(): object;
}

export class Repositorio<T> extends IEntidad {
  private items: Map<string, T> = new Map();
  private nombreEntidad: string;

  constructor(nombreEntidad: string) { this.nombreEntidad = nombreEntidad; }

  agregar(item: T): void {
    if (this.items.has(item.getId())) {
      throw new Error(`${this.nombreEntidad} con ID ${item.getId()} ya existe`);
    }
    this.items.set(item.getId(), item);
    this.guardarEnStorage();
  }

  obtener(id: string): T {
    const item = this.items.get(id);
    if (!item) throw new Error(`${this.nombreEntidad} con ID ${id} no encontrado`);
    return item;
  }

  obtenerTodos(): T[] { return Array.from(this.items.values()); }
  contar(): number { return this.items.size; }

  private guardarEnStorage(): void {
    const datos = Array.from(this.items.values()).map(i => i.toJSON());
    localStorage.setItem(`crm_${this.nombreEntidad}`, JSON.stringify(datos));
  }
}
```

Figura 7. `src/services/Repositorio.ts` — clase genérica con persistencia en `localStorage` (fragmento).

5.1. Reutilización con distintas entidades

Una sola implementación de `Repositorio<T>` se instancia con tres tipos concretos distintos a lo largo del sistema, cada uno con su propio espacio de almacenamiento en `localStorage`:

```
const repoClientes = new Repositorio<Cliente>('clientes');
```

```
const repoEmpleados = new Repositorio<Empleado>('empleados');
const repoServicios = new Repositorio<Servicio>('servicios');

repoClientes.agregar(new Cliente('c1', 'Maria Lopez', '70012345', 'maria@email.com'));
repoEmpleados.agregar(new Empleado('e1', 'Pedro Mamani', '77001122', 'pedro@crm.com',
    [Especialidad.CORTE, Especialidad.BARBERIA]));
repoServicios.agregar(new Servicio('s1', 'Corte', 50, 30, CategoriaServicio.CORTE));
```

Figura 8. Ejemplo de uso de Repositorio<T> con Cliente, Empleado y Servicio.

Gracias a la restricción `extends IEntidad`, el compilador de TypeScript garantiza en tiempo de compilación que solo se puedan almacenar objetos que expongan `getId()` y `toJSON()`, evitando errores en tiempo de ejecución por entidades incompatibles.

6. Manejo de Excepciones

El sistema aplica un manejo defensivo de excepciones basado en `throw new Error(...)` dentro de constructores y métodos de mutación de estado, evitando que se creen o modifiquen objetos en un estado inválido. A continuación se listan las validaciones más relevantes.

6.1. Validaciones en constructores

Clase	Validación	Mensaje de error
Persona	El id no puede estar vacío	'El ID no puede estar vacío'
Persona	No se puede instanciar directamente (<code>new.target</code>)	'Persona es una clase abstracta'
Cliente	El nombre y el teléfono no pueden estar vacíos	'El nombre no puede estar vacío' / 'El teléfono no puede estar vacío'
Empleado	Debe tener al menos una especialidad	'El empleado debe tener al menos una especialidad'
Servicio	El precio no puede ser negativo ni superar el máximo	'El precio no puede ser negativo' / 'El precio no puede superar 10000'
Servicio	La duración debe ser mayor a 0	'La duración debe ser mayor a 0'
Cita	<code>clienteId</code> , <code>empleadoId</code> y <code>servicioId</code> no pueden estar vacíos	'El <code>clienteId</code> no puede estar vacío' (y análogos)

```
constructor(id: string, nombre: string, precio: number, duracion: number,
  categoria: CategoriaServicio) {
  if (precio < 0) throw new Error('El precio no puede ser negativo');
  if (precio > Servicio.PRECIO_MAXIMO)
    throw new Error(`El precio no puede superar ${Servicio.PRECIO_MAXIMO}`);
  if (duracion <= 0) throw new Error('La duración debe ser mayor a 0');
  if (!nombre.trim()) throw new Error('El nombre no puede estar vacío');
  /* ... asignación de atributos ... */
}
```

Figura 9. `src/models/Servicio.ts` — validación de reglas de negocio en el constructor.

6.2. Excepciones en transiciones de estado (Cita)

La clase `Cita` modela una máquina de estados simple (PENDIENTE → CONFIRMADA → COMPLETADA, con CANCELADA como salida alternativa) y cada método de transición valida que el cambio solicitado sea coherente con el estado actual antes de aplicarlo, lanzando una excepción en caso contrario.

```
confirmar(): void {
  if (this.estado === EstadoCita.CANCELADA)
    throw new Error('No se puede confirmar una cita cancelada');
  this.estado = EstadoCita.CONFIRMADA;
```

```
}

cancelar(): void {
  if (this.estado === EstadoCita.COMPLETADA)
    throw new Error('No se puede cancelar una cita completada');
  this.estado = EstadoCita.CANCELADA;
}

completar(): void {
  if (this.estado !== EstadoCita.CONFIRMADA)
    throw new Error('Solo se pueden completar citas confirmadas');
  this.estado = EstadoCita.COMPLETADA;
}
```

Figura 10. *src/models/Cita.ts* — transiciones de estado protegidas con excepciones.

El Repositorio<T> aplica el mismo principio a nivel de persistencia, lanzando excepciones ante intentos de agregar un identificador duplicado, actualizar una entidad inexistente o eliminar un elemento que no está registrado, tal como se ilustra en la Figura 7.

7. Pruebas Unitarias

La verificación del sistema se implementa en `src/tests/Repositorio.test.ts` mediante un pequeño framework de pruebas propio (funciones `test()`, `expect()` y `expectThrow()`), ejecutado con `ts-node` a través del script `npm test`. El archivo redefine localmente las clases del dominio para operar en un entorno Node.js sin DOM, sustituyendo `localStorage` por un objeto en memoria. Las pruebas se organizan en cinco suites temáticas:

Suite	Enfoque	Casos representativos
1. Herencia, Polimorfismo y Clases Abstractas	Verifica que Cliente y Empleado sean instancias de Persona y que <code>getRol()/obtenerFichaTecnica()</code> difieran por subclase.	6 pruebas
2. Enums del Sistema	Valida los valores de <code>EstadoCita</code> y <code>TipoCliente</code> , y la evolución automática de tipo de cliente (Nuevo→Regular→VIP).	5 pruebas
3. Transiciones de Estado de Cita	Cubre el flujo PENDIENTE→CONFIRMADA→COMPLETADA y las excepciones ante transiciones inválidas.	6 pruebas
4. Métodos Estáticos y Validaciones	Prueba <code>compararPorPrecio()</code> , <code>PRECIO_MAXIMO</code> y las excepciones de validación de Servicio, Empleado y Cita.	6 pruebas
5. Repositorio<T> Genérico	Verifica <code>agregar/obtener/eliminar/contar</code> con Cliente, Empleado y Servicio, y las excepciones por ID duplicado o inexistente.	6 pruebas

7.1. Utilidades de prueba

El propio archivo de pruebas define un mini-framework con las funciones `test()`, `expect()` y `expectThrow()`, que permite ejecutar los casos y reportar resultados sin depender de librerías externas como Jest:

```
function test(nombre: string, fn: () => void): void {
  try { fn(); console.log(`  ✔️ ${nombre}`); passed++; }
  catch (e: any) { console.log(`  ❌ ${nombre}\n    ${e.message}`); failed++; }
}

function expectThrow(fn: () => void, msg?: string): void {
  try { fn(); throw new Error('Se esperaba excepción pero no se lanzó'); }
  catch (e: any) {
    if (e.message === 'Se esperaba excepción pero no se lanzó') throw e;
    if (msg && !e.message.includes(msg))
      throw new Error(`Esperaba "${msg}", obtuvo "${e.message}"`);
  }
}
```

Figura 11. `src/tests/Repositorio.test.ts` — utilidades `test()/expectThrow()` del framework propio.

7.2. Prueba significativa: transición de estado inválida

El siguiente caso pertenece a la Suite 3 y verifica que el sistema impida completar una cita que no ha sido confirmada previamente, un ejemplo directo de la relación entre manejo de excepciones y consistencia del dominio:

```
test('Excepcion: no se puede completar cita sin confirmar', () => {
  const c = new Cita('c4', 'cli1', 'empl', 'srv1', new Date());
  expectThrow(() => c.completar(), 'Solo se pueden completar citas confirmadas');
});

test('Repositorio lanza error en ID duplicado', () => {
  const repo = new Repositorio<Cliente>('cli2');
  repo.agregar(new Cliente('dup', 'Test', '111', 't@t.com'));
  expectThrow(() => repo.agregar(new Cliente('dup', 'Test2', '222', 't2@t.com')), 'ya
existe');
});
```

Figura 12. `src/tests/Repositorio.test.ts` — casos de prueba de excepciones (Suite 3 y Suite 5).

Al finalizar la ejecución, el script imprime un resumen consolidado con el número de pruebas superadas y fallidas, y retorna un código de salida distinto de cero si alguna prueba falla, lo que permite integrarlo fácilmente en un flujo de integración continua.

8. Estructura del Proyecto

El repositorio sigue una organización simple orientada a separar el dominio (models), la infraestructura de persistencia (services), las pruebas (tests) y la interfaz visual (public):

```
crm-final-sis113/
|-- src/
|   |-- models/
|   |   |-- Persona.ts           # Clase abstracta base (Persona)
|   |   |-- Cliente.ts          # Hereda de Persona + enum TipoCliente
|   |   |-- Empleado.ts         # Hereda de Persona + enum Especialidad
|   |   |-- Servicio.ts         # Implementa IServicio + enum CategoriaServicio
|   |   |-- Cita.ts             # Enum EstadoCita + transiciones de estado
|   |   |-- `-- index.ts        # Demostraciones de POO (polimorfismo, enums, etc.)
|   |-- services/
|   |   |-- `-- Repositorio.ts   # Clase genérica Repositorio<T> + interfaz IEntidad
|   |-- `-- tests/
|   |   |-- `-- Repositorio.test.ts # 5 suites de pruebas unitarias
|-- public/
|   |-- `-- index.html           # Interfaz visual completa (Dashboard, Clientes,
|   |                               # Citas, Servicios, Empleados, Calendario)
|-- docs/
|   |-- `-- uml_crm_citass.png   # Diagrama UML de clases
|-- tsconfig.json               # Configuración del compilador de TypeScript
|-- package.json                # Scripts (build, watch, test, demo) y dependencias
|-- `-- README.md               # Documentación general del proyecto
```

Figura 13. Árbol de directorios del repositorio crm-final-sis113.

8.1. Propósito de cada carpeta

Carpeta / archivo	Propósito
src/models/	Contiene las clases de dominio (entidades) y sus enums asociados: Persona, Cliente, Empleado, Servicio y Cita, además del archivo index.ts que ejecuta demostraciones de los conceptos POO aplicados.
src/services/	Contiene la capa de persistencia genérica: la clase Repositorio<T> y la interfaz IEntidad que deben cumplir las entidades almacenables.
src/tests/	Contiene la suite de pruebas unitarias del proyecto, con su propio framework de aserciones.
public/	Contiene la interfaz web (index.html) que consume la lógica de dominio compilada, con las vistas de Dashboard, Clientes, Citas, Servicios, Empleados y Calendario.
docs/	Material de apoyo del proyecto, incluyendo el diagrama UML de clases.
tsconfig.json / package.json	Configuración de compilación de TypeScript y definición de scripts npm (build, watch, test, demo).

9. Instrucciones de Ejecución

El proyecto no requiere backend ni base de datos: basta con Node.js y npm para compilar y probar el código TypeScript, y un navegador web para ejecutar la interfaz. Los pasos siguientes reproducen el flujo descrito en el README.md del repositorio.

9.1. Clonar el repositorio

```
git clone https://github.com/FabricioRivero/crm-final-sis113.git
cd crm-final-sis113
```

9.2. Instalar dependencias

```
npm install
```

Esto instala las dependencias de desarrollo declaradas en package.json: typescript, ts-node y @types/node.

9.3. Compilar el proyecto

```
npm run build      # Ejecuta tsc y genera el JavaScript compilado
npm run watch      # Igual que build, pero en modo observación (recompila al guardar)
```

9.4. Ejecutar la suite de pruebas

```
npm test
```

Este comando ejecuta npx ts-node --transpile-only src/tests/Repositorio.test.ts, corriendo las cinco suites descritas en la Sección 7 y mostrando en consola el resultado de cada caso junto con el resumen final de pruebas superadas y fallidas.

9.5. Ejecutar la demostración de consola

```
npm run demo
```

Este comando ejecuta src/models/index.ts, imprimiendo en consola las demostraciones de polimorfismo, enums, métodos estáticos y transiciones de estado de una Cita, incluyendo la captura de una excepción en tiempo real.

9.6. Abrir la interfaz visual

1. Abrir el archivo public/index.html directamente en el navegador (doble clic, o arrastrarlo a una pestaña nueva).
2. El sistema carga automáticamente datos de ejemplo (clientes, empleados, servicios y citas) para su exploración inmediata.

3. Navegar entre los módulos Dashboard, Clientes, Citas, Servicios, Empleados y Calendario desde el menú lateral.

10. Capturas de Pantalla (Interfaz)

La interfaz visual (public/index.html) se organiza en seis vistas navegables desde un menú lateral, tituladas “ CRM Citas”. A continuación se describe el contenido esperado de cada captura de pantalla del sistema en funcionamiento.

10.1. Dashboard

Vista de inicio del sistema. Muestra un panel de estadísticas generales del negocio (número de clientes registrados, citas activas, ingresos estimados por servicios, etc.) junto con un listado de las próximas citas programadas, permitiendo al usuario obtener una visión general del estado del negocio sin necesidad de navegar a otros módulos.

10.2. Clientes

Vista de tipo listado/tabla con el registro completo de clientes. Incluye buscador, datos de contacto (teléfono, correo electrónico), la etiqueta de tipo de cliente (Nuevo, Regular o VIP) calculada automáticamente según el número de citas en su historial, y acceso al detalle de cada cliente.

10.3. Citas

Vista central del sistema, donde se listan las citas registradas junto con su estado (pendiente, confirmada, cancelada o completada) representado mediante etiquetas de color. Permite crear nuevas citas seleccionando cliente, empleado y servicio, así como ejecutar las acciones de confirmar, cancelar o completar sobre cada cita.

10.4. Calendario

Vista de calendario mensual con navegación entre meses (anterior/siguiente). Cada día del mes muestra las citas programadas para esa fecha, permitiendo al usuario visualizar de forma espacial la carga de trabajo del negocio a lo largo del mes.

Nota sobre las capturas

Las imágenes reales de cada vista deben insertarse en esta sección al exportar el informe final, tomando pantallazos de public/index.html abierto en el navegador tras cargar los datos de ejemplo.

11. Desafíos y Soluciones

Durante el desarrollo del proyecto, el equipo enfrentó distintos desafíos técnicos propios de aplicar POO estricta en TypeScript sobre un dominio con reglas de negocio y persistencia local. La siguiente tabla resume los principales desafíos y las soluciones adoptadas.

Área	Desafío	Solución adoptada
TypeScript	Impedir la instanciación directa de la clase abstracta Persona, ya que TypeScript no elimina las clases abstractas en tiempo de ejecución (JavaScript no las conoce nativamente).	Verificación explícita de <code>new.target === Persona</code> dentro del constructor, lanzando una excepción si se intenta instanciar la clase base directamente.
Persistencia	No contar con un backend ni base de datos para conservar la información entre sesiones del usuario.	Se optó por <code>localStorage</code> del navegador como mecanismo de persistencia, encapsulado íntegramente dentro de <code>Repositorio<T></code> mediante <code>guardarEnStorage()/cargarDesdeStorage()</code> , de modo que el resto del sistema desconoce el mecanismo concreto de almacenamiento.
Genéricos	Evitar duplicar la lógica de CRUD (agregar, obtener, actualizar, eliminar) para cada entidad (Cliente, Empleado, Servicio).	Diseño de una única clase <code>Repositorio<T extends IEntidad></code> , restringida mediante un <code>generic</code> acotado a la interfaz mínima <code>IEntidad</code> , reutilizable para cualquier entidad futura del sistema.
Validaciones	Garantizar que las entidades y las transiciones de estado de una Cita nunca queden en un estado inconsistente (por ejemplo, completar una cita nunca confirmada).	Validaciones exhaustivas con <code>throw new Error()</code> en constructores y en cada método de transición de estado, cubiertas además por pruebas unitarias específicas (Suites 3 y 4).

12. Conclusiones

Luis Fabricio Rivero Aban

El desarrollo del módulo de interfaz y de la clase Persona me permitió comprender en la práctica por qué TypeScript, a diferencia de JavaScript puro, resulta valioso para modelar dominios con reglas estrictas: el sistema de tipos detecta en tiempo de compilación errores que en un CRM real se traducirían en datos corruptos, y el trabajo con `abstract` y `new.target` reforzó la diferencia entre abstracción como concepto de diseño y su implementación real en un lenguaje que compila a JavaScript.

Santiago Antonio Encinas Jadue

Trabajar en los modelos y la lógica de negocio (Cliente, Empleado, Cita) me permitió aplicar de forma concreta la herencia y el polimorfismo revisados en la materia, especialmente al diseñar las transiciones de estado de una cita como una pequeña máquina de estados protegida por excepciones. Esto reforzó la importancia de validar los datos en el punto exacto donde se modifican, en lugar de confiar en validaciones externas dispersas.

Santiago Zamora Barrios

Mi aporte se centró en los servicios y la interfaz de usuario, donde pude comprobar cómo el diseño genérico del `Repositorio<T>` simplificó enormemente la persistencia: una sola clase resolvió el almacenamiento de tres entidades distintas sin duplicar código. El proyecto en conjunto consolidó mi comprensión de cómo los principios de POO se traducen en un sistema mantenible y extensible en un contexto real de negocio.

Conclusión general

El proyecto CRM de Citas para Negocios cumple con los objetivos planteados para la materia Programación II: aplica de forma verificable abstracción, herencia, polimorfismo, encapsulamiento, interfaces y genéricos, respaldados por manejo de excepciones y una suite de pruebas unitarias propia. El resultado es un sistema funcional, sin dependencias de backend, que demuestra que los principios de POO revisados en clase son directamente aplicables a la resolución de un problema de negocio real como la gestión de citas.

13. Referencias Bibliográficas

Deitel, H. M., & Deitel, P. J. (2016). *C++ cómo programar* (9.^a ed.). Pearson Educación.

Vanderkam, D. (2019). *Effective TypeScript: 62 Specific Ways to Improve Your TypeScript*. O'Reilly Media.

Khorikov, V. (2020). *Unit Testing Principles, Practices, and Patterns*. Manning Publications.

TypeScript. (2026). *Documentación oficial de TypeScript*. <https://www.typescriptlang.org>